

Computational Physics_ Dr. Ejtehad

Homework03_Percolation

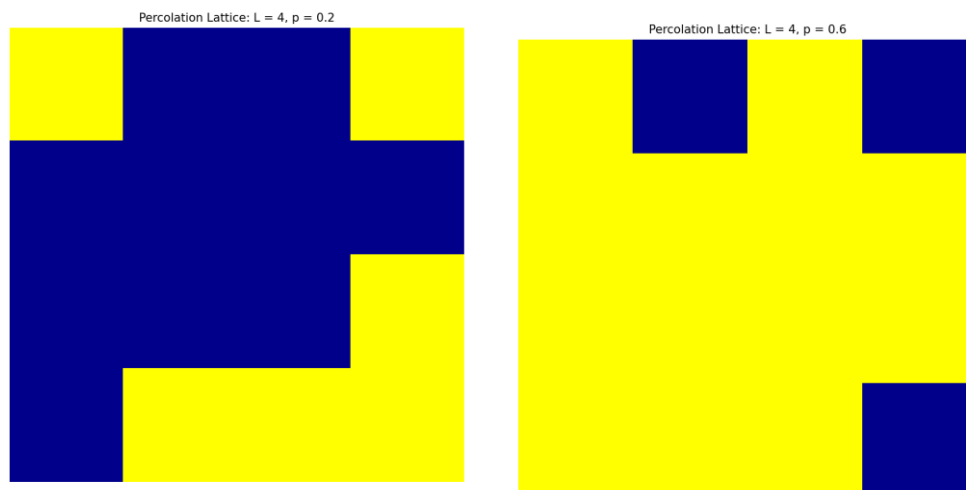
Mahsa Rahimi_99100671

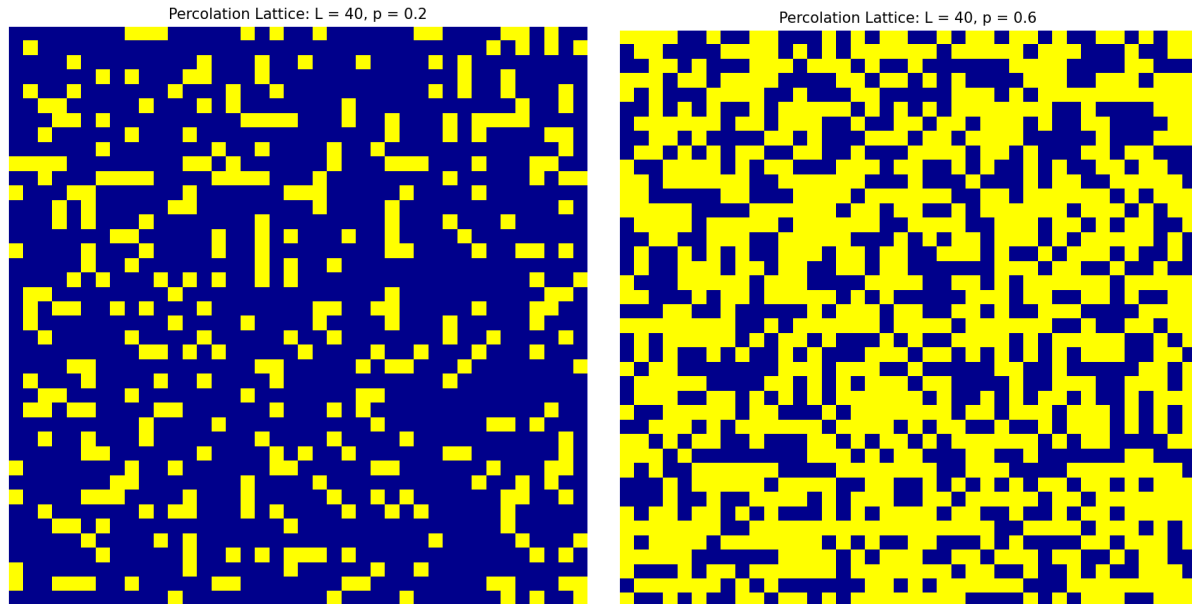
1- Percolation

General Idea: The code implements a simple site percolation model on a two-dimensional square lattice of size $L \times L$. Each site in the lattice can be occupied or unoccupied, determined by a probability p . In percolation theory, one is often interested in whether there exists a path of adjacent occupied sites that spans the lattice from one side to the other. In this particular example, the code checks for a path connecting the left boundary to the right boundary of the grid. If such a path exists, the system is said to percolate (or, equivalently, to have a spanning cluster).

Algorithm and Approach: The program begins by prompting the user for two inputs: the lattice size L and the occupation probability p . Once these values are provided, it generates an $L \times L$ array, where each site is determined to be occupied if a randomly drawn number is less than p . This step effectively creates a two-dimensional Boolean lattice, with True indicating an occupied site and False indicating an unoccupied site. To determine whether a spanning cluster exists, the code scans all sites along the left edge of the lattice. For each site that is occupied, it initiates a depth-first search (DFS), exploring the lattice one step at a time. This search examines the site's immediate neighbours (above, below, left, and right), adding them to the exploration queue if they are both occupied and unvisited. The DFS continues spreading outward in this fashion, marking each site as visited to avoid repeated checks. If, at any point, the search encounters a site on the right edge, it concludes that a spanning path of occupied sites exists, and the code prints "1" to signal percolation. If no such path is found after examining every occupied site on the left boundary, the program prints "0". Finally, the program displays the generated lattice in two colours—one for occupied sites (yellow) and one for unoccupied sites (blue).

Results and Visualisation:



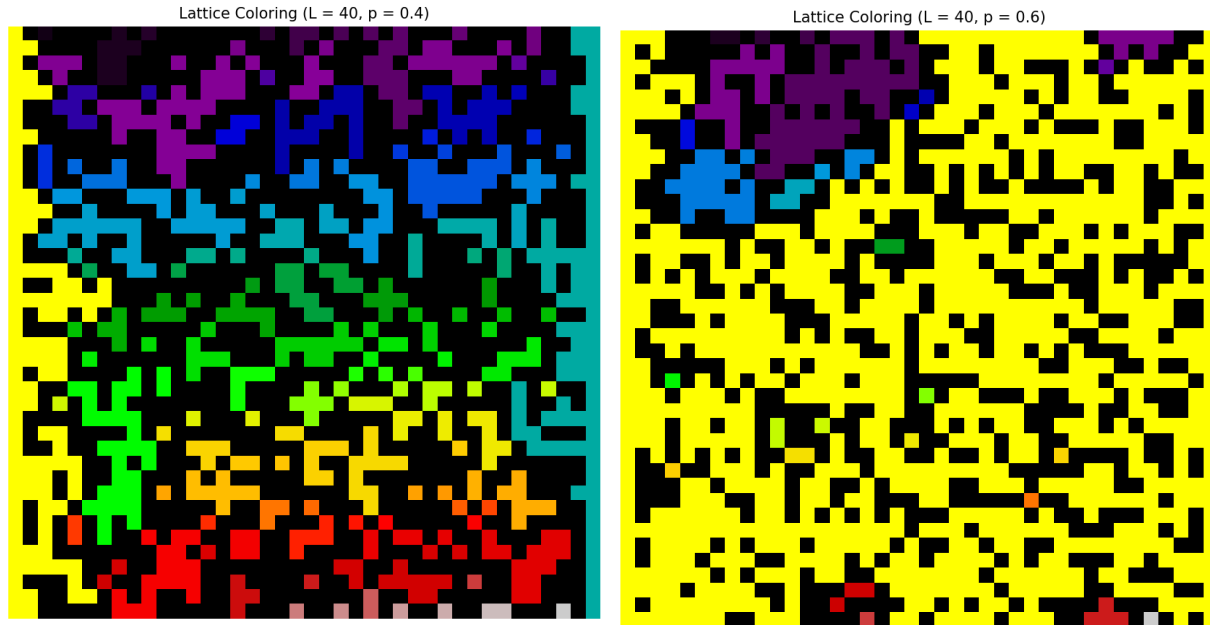


2- Flood-Fill Algorithm

General Idea: This code expands upon the concept of site percolation by labelling and visualising all clusters in a randomly generated $L \times L$ lattice. Each site is occupied with probability p . Clusters are identified using a flood-fill approach, where adjacent occupied sites are assigned a common label. The program then checks whether any labelled cluster spans the lattice from the left edge to the right edge—indicating percolation—and highlights such clusters in a distinct colour.

Algorithm and Approach: The program begins by prompting the user for two inputs: the lattice size L and the occupation probability p . It then creates an $L \times L$ Boolean lattice by generating random numbers at each site and comparing them to p . Any site whose random value is less than p is marked as occupied. Once the lattice is initialised, the code labels clusters of adjacent occupied sites. To do this, it loops over every site in the grid, looking for an unlabelled occupied site. Upon finding one, it initiates a flood-fill process that assigns a unique label (cluster ID) to all connected sites. Connectedness is determined by up, down, left, and right adjacency. Each time the flood-fill is called, it uses a stack to explore every reachable occupied site, labelling them with the current cluster ID . After the entire connected region is labelled, the cluster ID is incremented, and the process continues until all occupied sites have been assigned to clusters. With all clusters labelled, the program checks for percolation by identifying any label present on both the left edge and the right edge of the lattice. If a label appears on both edges, it indicates a percolating cluster that spans the system. The code prints “1” if any such label is found (and lists the label(s) that percolate), or “0” otherwise. Finally, a visualisation step displays each cluster in a different colour. If a cluster is determined to be percolating, it is highlighted in yellow. This colour map thus provides an immediate visual confirmation of whether percolation has occurred, while also showing the distribution of all other clusters in the lattice. Additionally, the code includes a timing mechanism that measures and reports the total execution time, allowing for direct performance comparisons with other methods.

Results and Visualisation:

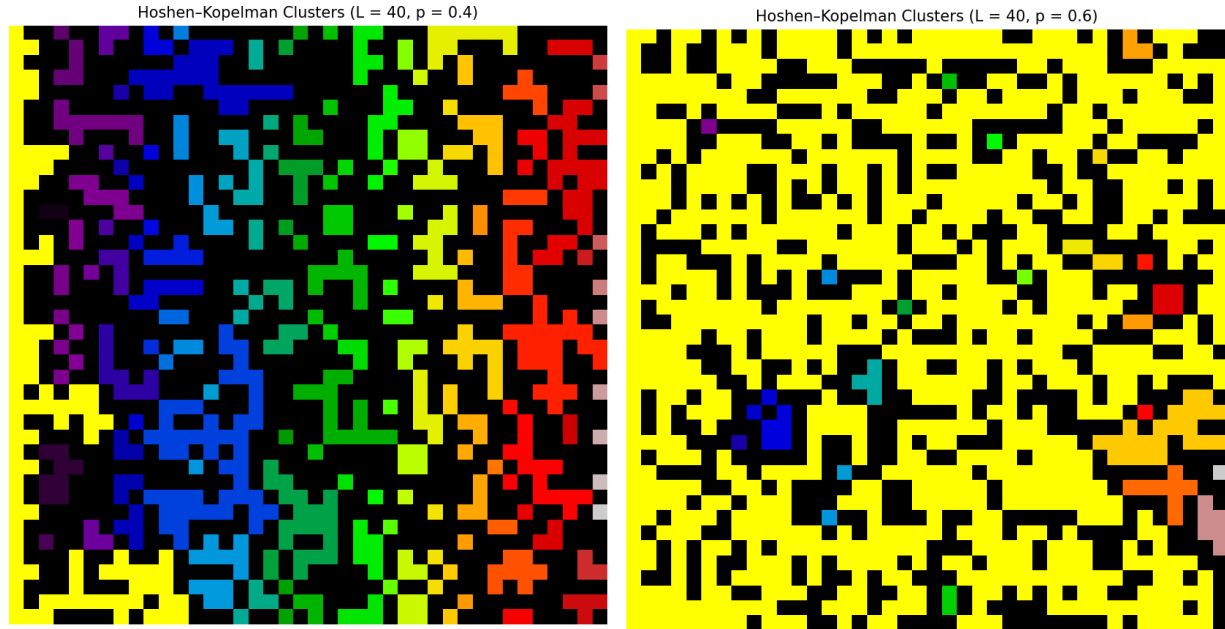


3- Hoshen-Kopelman Algorithm

General Idea: Here the Hoshen-Kopelman algorithm is demonstrated for cluster labelling in a percolation setting. Unlike a simple flood-fill approach, Hoshen-Kopelman is designed to label all clusters in a single pass across the lattice. It also uses a union-find data structure to keep track of cluster equivalences. Here, the left boundary of the lattice is forced to be fully occupied (label 1), and all other sites are occupied with probability p . If any site on the right boundary ends up sharing label 1, it indicates that a percolating path exists from left to right.

Algorithm and Approach: Instead of performing repeated flood-fill searches for every unvisited occupied site, the Hoshen-Kopelman algorithm processes the entire lattice in one continuous pass from left to right and top to bottom. In our implementation, we start by forcing the left boundary of the lattice to be occupied and immediately label it with 1. All other sites are then set to be occupied with a probability p . As the algorithm scans each occupied site, it examines the top and left neighbours. If both neighbours are unoccupied, the site is assigned a new label; if one neighbour is occupied, the site inherits that neighbour's label; and if both are occupied, the algorithm merges their labels using a union-find data structure. This merging step is key because it avoids the overhead of multiple flood-fill operations by immediately consolidating connected clusters as they are discovered. After the entire lattice is processed, a second pass compresses all labels to their root representatives, ensuring that every site in a connected cluster has the same final label. Finally, the algorithm checks if label 1, which represents the forced left boundary, appears on the right boundary, thereby indicating the presence of a percolating cluster. This method is more efficient and scalable compared to the repeated flood-fill approach, especially for large lattices, because it minimises redundant searches and streamlines the cluster identification process.

Results and Visualisation:



Over a series of tests on a 100×100 lattice at different occupation probabilities ($p = 0.3, 0.6, 0.9$), the Flood-Fill algorithm consistently required more time than Hoshen-Kopelman. For instance, Flood-Fill took approximately 0.0267 seconds at $p = 0.3$, 0.0278 seconds at $p = 0.6$, and 0.0345 seconds at $p = 0.9$. In contrast, Hoshen-Kopelman completed the same tasks in around 0.0190, 0.0262, and 0.0220 seconds, respectively. We also ran the same tests on a 200×200 lattice, where the time difference between the two methods became more pronounced. Flood-Fill completed its runs in approximately 0.0477 seconds at $p = 0.3$, 0.0728 seconds at $p = 0.6$, and 0.0909 seconds at $p = 0.9$. Meanwhile, Hoshen-Kopelman took only about 0.0394, 0.0570, and 0.0615 seconds for those same probabilities.

The primary reason for these differences lies in the way each algorithm identifies clusters. Flood-Fill initiates a new search each time it encounters an unvisited occupied site, which can lead to repeated traversals of the same regions. Hoshen-Kopelman, on the other hand, labels clusters in a single pass by merging connected sites on the fly using a union-find data structure. This approach avoids redundant searches and scales more efficiently, especially as the lattice size or site occupancy increases.

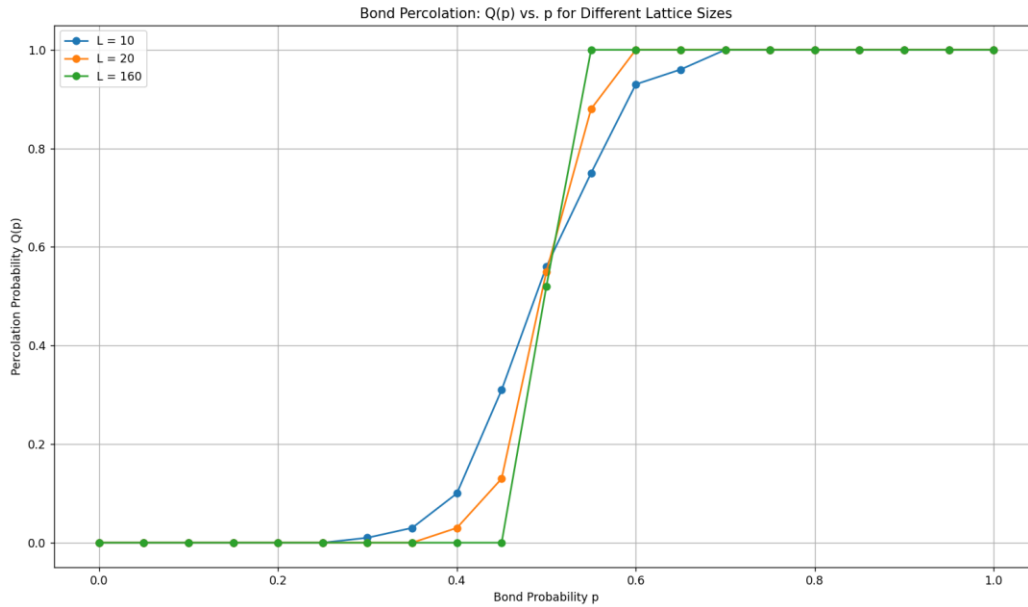
4- Finite Lattice 01

General Idea: This exercise implements bond percolation on a $2D$ square lattice to study how the probability of forming an infinite cluster (Q) depends on the bond probability p and the lattice size L . In percolation theory, Q represents the likelihood that a connected path of open bonds spans the system, transitioning from an “insulating” state (no spanning path) to a “conducting” state (spanning path exists) near the critical probability p_c . Although the transition is abrupt in the thermodynamic limit ($L \rightarrow \infty$), finite lattices exhibit a smoother transition. To capture reliable

statistics, we run multiple simulations for each probability p and take the average of Q . The code then plots $Q(p)$ for different lattice sizes, illustrating how larger systems approach the sharp transition characteristic of an infinite lattice.

Algorithm and Approach: To investigate bond percolation on square lattices of various sizes ($L = 10, 20, 40, 80, 160$), the program scans the bond probability p in increments of $\Delta p = 0.05$, starting from $p = 0$ up to $p = 1$. For each value of p , it performs 100 independent simulations, ensuring a reliable statistical estimate of the percolation probability $Q(p)$. In each simulation, every site in an $L \times L$ lattice is assigned a unique index, and bonds between neighbouring sites (up-down and left-right) are declared open with probability p . Whenever a bond is open, the algorithm unites the two corresponding sites in a union-find (disjoint-set) structure, effectively merging their clusters into a single connected component. After all potential bonds have been considered, the code identifies which clusters lie on the left boundary (column 0) and which lie on the right boundary (column $L - 1$). If there is a common cluster between these boundaries, the system is deemed to have percolated. This process is repeated 100 times for each p , and $Q(p)$ is computed as the fraction of simulations that show a spanning cluster. The results are then plotted for several lattice sizes (10, 20, 160), focusing on how $Q(p)$ evolves with p .

Results and Visualisation:



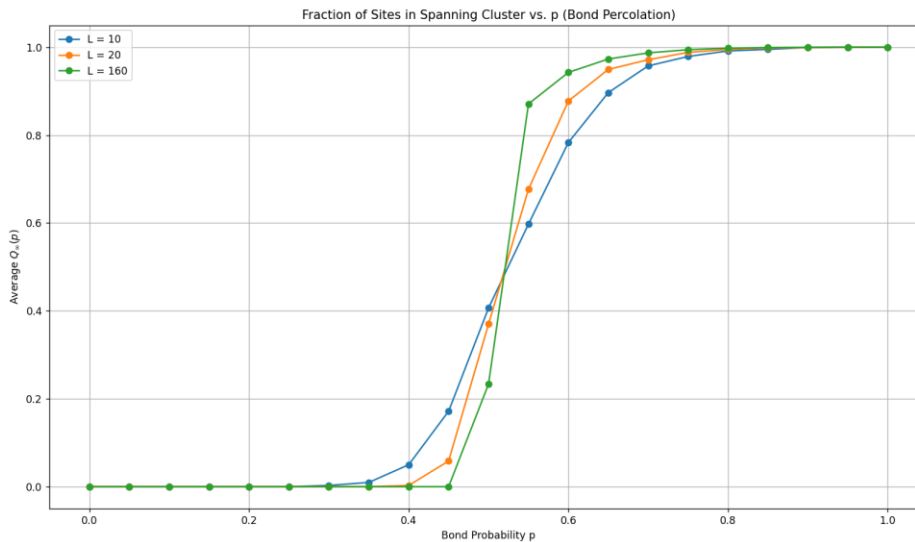
The plotted curves show the probability of forming an infinite cluster $Q(p)$ as a function of the bond probability p . For small values of p , the probability of percolation is effectively zero, indicating that the system is below the percolation threshold. As p increases, there is a sharp transition where $Q(p)$ rapidly rises from near zero to one. This transition becomes steeper as the lattice size increases, which aligns with the expectation that in the thermodynamic limit, percolation behaves as a phase transition. For smaller lattices, the transition is more gradual due to finite-size effects, whereas for larger lattices, the transition occurs more abruptly near the critical probability p_c .

5- Finite Lattice 02

General Idea: Building on the previous bond percolation approach—where we only checked whether a percolating cluster existed—here we go one step further by quantifying the size of that cluster. Instead of a simple “yes or no” outcome, it calculates the fraction of lattice sites that belong to any spanning cluster, denoted Q_∞ . By averaging this fraction over many simulations for each probability p , we gain a deeper understanding of how large the percolating component can become, especially near the percolation threshold.

Algorithm and Approach: As before, we represent each site in an $L \times L$ lattice with a union-find (disjoint-set) structure. Adjacent sites are bonded with probability p , and each open bond unites the two sites in the union-find data structure, merging their clusters. This time, every cluster tracks its size, i.e., how many sites it contains. After assigning all possible bonds, we identify clusters that span from the left edge to the right edge. If such clusters exist, we sum up their sizes and divide by $N = L^2$ to obtain Q_∞ , the fraction of sites belonging to the percolating component. Because finite systems exhibit statistical fluctuations, the simulation is repeated multiple times (e.g., 100 runs) for each probability p , and the average Q_∞ is computed. Scanning through values of p in increments of 0.05 and plotting the results for different lattice sizes ($L = 10, 20, 160$) reveals how larger systems display a sharper rise in Q_∞ near the critical region, while smaller systems show a smoother transition.

Results and Visualisation:



From the figure, we see that for small bond probabilities p , the fraction of sites in the spanning cluster Q_∞ remains near zero, meaning that any percolating cluster—if it exists at all—is too small to occupy a significant portion of the lattice. As p increases, Q_∞ rises rapidly from near zero to values approaching one. This transition becomes sharper as the lattice size L grows, reflecting how larger systems better approximate the discontinuous phase transition seen in the thermodynamic limit.

For smaller lattices (e.g., $L = 10$), the transition region is broader, indicating finite-size effects that smooth out the abrupt jump. By contrast, the largest lattice ($L = 160$) shows a more pronounced rise in Q_∞ , consistent with the system crossing a percolation threshold where an extensive fraction of sites suddenly becomes part of a single spanning cluster. As p approaches 1, virtually all bonds are open, and the spanning cluster covers nearly the entire lattice, causing Q_∞ to saturate close to unity.

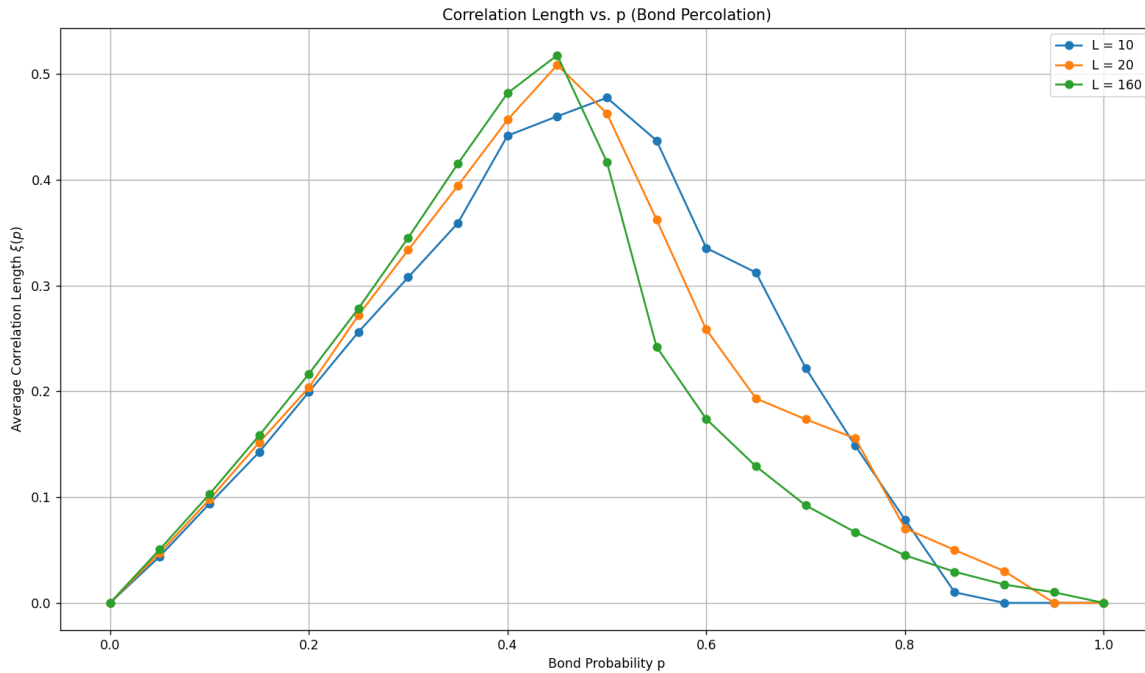
6- Correlation Length

General Idea: This code extends the previous bond percolation model to measure the average correlation length ξ , which is derived from the radius of gyration R_g of each finite cluster. The radius of gyration offers a more nuanced measure of cluster size than simply counting the number of occupied sites, since it incorporates the spatial distribution of the cluster's sites. By excluding the spanning (infinite) cluster—i.e., clusters connecting the left and right boundaries—the program focuses on the size of finite clusters, whose radius of gyration collectively defines the correlation length. As p approaches the critical probability p_c , these finite clusters grow larger, and ξ increases sharply, reflecting the incipient divergence of cluster size at the threshold.

Algorithm and Approach: In this program, each site in an $L \times L$ lattice is represented by a node in a union-find (disjoint-set) data structure, and every pair of adjacent sites (horizontal or vertical) is connected with probability p . Whenever a bond is open, the two corresponding nodes are united, merging their clusters. In addition to maintaining each cluster's size, the code also tracks the sums of the x - and y -coordinates of its sites and the sums of their squares. These accumulated sums allow for the calculation of the radius of gyration for each finite cluster, which is a structural measure of how spread out the cluster's sites are.

Because the goal is to measure the correlation length ξ , which reflects the average size of finite clusters, the program excludes any cluster that spans from the left edge to the right edge of the lattice (i.e., the infinite or percolating cluster). Once all finite clusters have been identified, the code computes each cluster's radius of gyration and takes the average R_g across those clusters to obtain ξ . To account for statistical fluctuations, multiple simulations (e.g., 50 runs) are performed at each bond probability p . By repeating this procedure for different values of p (in increments of 0.05) and for various lattice sizes, the program produces a plot of ξ versus p . As p approaches the percolation threshold, clusters grow larger, causing ξ to increase sharply. This behavior is consistent with the idea that, in the thermodynamic limit, ξ diverges at the critical point, signifying the emergence of an infinite cluster.

Results and Visualisation:



From this plot, we see that the correlation length ξ starts off small at low bond probabilities p , where open bonds are sparse and clusters remain limited in size. As p increases, ξ rises sharply, reaching a maximum near the critical region. This peak reflects the fact that finite clusters grow larger just before an “infinite” cluster spans the lattice. Beyond that point, one large cluster dominates, and the finite clusters that remain are relatively small, causing ξ to decrease again.

As the size of the system increases, the value of p_c which is the point at which the graph reaches a peak, starts decreasing and gets shifted towards the left. (I though it would shift towards the right until it reaches the value of 0.5...? Hmmmmm.....)

One can calculate $p_c(\infty)$ from plotting the values we have for p_c (extracting them from the counsel, finding the peaks, i.e., the points that their ξ is the highest for a given probability p) for different L .

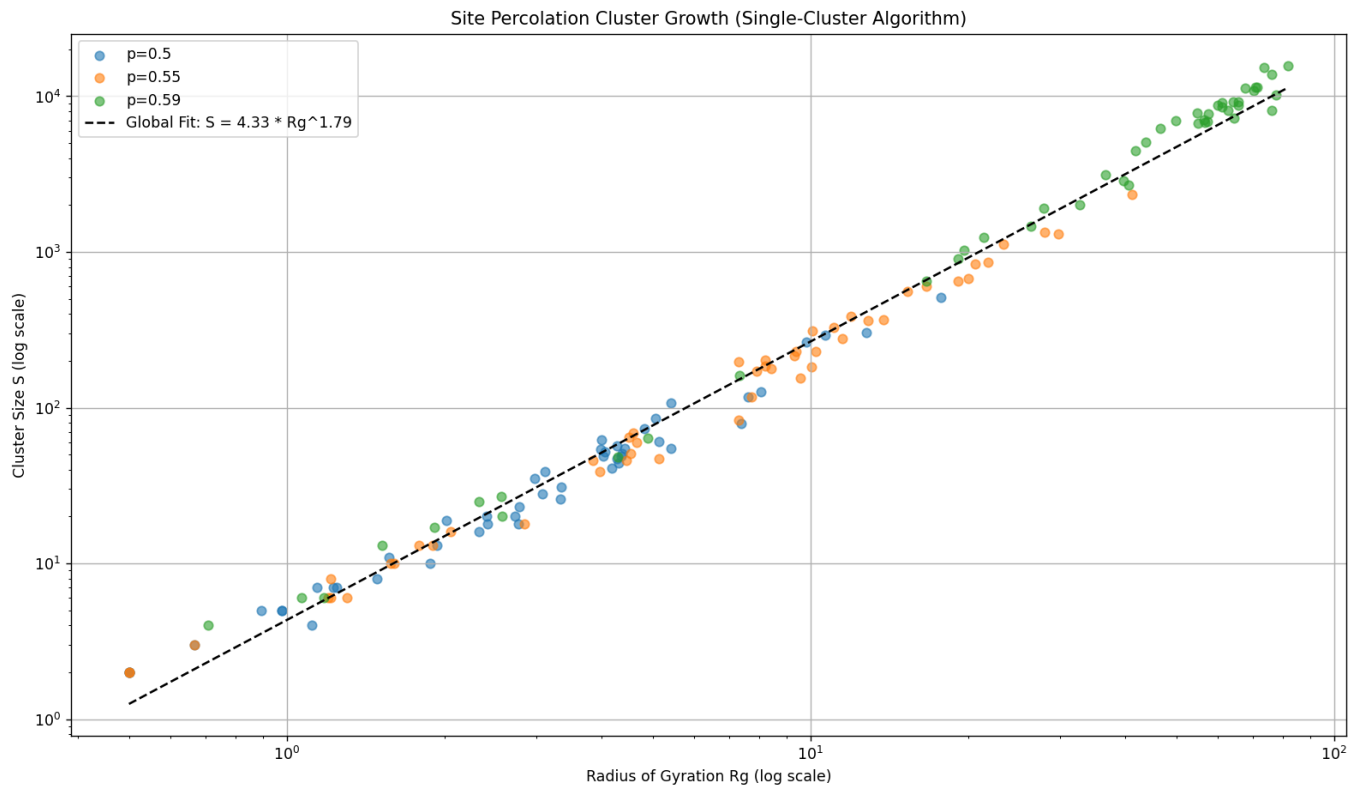
7- Grow Cluster

General Idea: This generates single clusters in a site percolation framework and analyses their structural properties. Instead of filling an entire lattice randomly, it grows one cluster at a time from a random seed site. Neighbours of each occupied site have a probability p of also becoming occupied, and this process continues until no further growth is possible. By recording the cluster size S and its radius of gyration R_g , we aim to reveal whether percolation clusters exhibit fractal characteristics.

Algorithm and Approach: In this single-cluster growth approach, the program first initialises an $L \times L$ lattice, setting every site to an “undecided” state (e.g., a value of 0). It then selects one site at random (or uses a given seed) to serve as the cluster’s origin. This site is designated “occupied” (value 1) and placed into a queue for processing. The cluster expands by repeatedly taking a site from the queue and examining its four neighbours (up, down, left, right), as long as they remain within the lattice boundaries. Whenever a neighbour is still “undecided,” the code draws a random number to decide whether the site becomes “occupied” (with probability p) or “blocked” (with probability $1 - p$). If it becomes occupied, that neighbour is added to the queue, enabling further growth. This process continues until the queue empties, indicating that no new neighbours can be incorporated into the cluster.

Once the growth phase ends, all sites that have been assigned “occupied” form a single cluster around the original seed. The cluster size S is simply the number of occupied sites, while the radius of gyration R_g is computed by finding the centre of mass of those sites and determining how far, on average, they lie from that centre. In practice, the code sums up the x - and y -coordinates of all occupied sites, along with their squares, and uses these sums to calculate R_g based on the difference between the mean of the squared distances and the square of the centre-of-mass coordinates. By running this procedure multiple times at each probability p , the code gathers data pairs (S, R_g) for each cluster. At the end, we plot these pairs on a log-log scale.

Results and Visualisation:



This log-log plot shows the cluster size S as a function of the radius of gyration R_g for all tested probabilities ($p = 0.50, 0.55, 0.59$). The data points are combined into a single dataset, and a global power-law fit is performed, yielding a best-fit line of the form $S = a \times R_g^b$. Here, a is a prefactor and b is the effective fractal dimension. The near-linear alignment of the data on a log-log scale suggests that the clusters exhibit fractal-like scaling, with the exponent b indicating how rapidly the cluster size grows as its radius increases. This single global fit highlights that, despite different percolation probabilities, the underlying geometry of the clusters follows a similar scaling relationship.